QWIM Dashboard and coding framework

Cristian Homescu

June 2026

Table of Contents

1 Setup

Recommended Python version Python 3.13 (version >= 3.13.10). The Python codes (including dashboard) were tested under Python version 3.13.13.

Recommended IDE: Visual Studio Code (VSC) version >= 1.121 with extensions (e.g. Python extensions)

To generate PDFs: install Typst compiler https://typst.app/open-source/

Recommended distributed version control system: Git

https://docs.github.com/en/get-started/git-basics

Recommended git branching approach: feature branches merged w/ “Pull Request” into “main” branch

https://gist.github.com/bryanbraun/8c93e154a93a08794291df1fcdce6918

https://docs.github.com/en/pull-requests

In VSC Terminal go to the folder obtained after unzipping the Viz-QWIM.zip

First update Python packages pip, uv and ruff

```
python -m pip install -U pip uv ruff
```

Create Python virtual environment using “uv”

```
uv add polars   (NOTE: uses pyproject.toml)
```

Activate the created Python virtual environment

```
.venv\Scripts\activate
```

Generate the baselines for regression tests

```
python -m tests.regression_data.generate_regression_baselines
```

Run tests

```
pytest tests -q
```

If you want to generate the unit testing coverage

```
pytest tests -q --cov=src --cov-report=term --cov-report=html --cov-report=json
```

Run the dashboard

```
shiny run --launch-browser src/dashboard/main_App.py
```

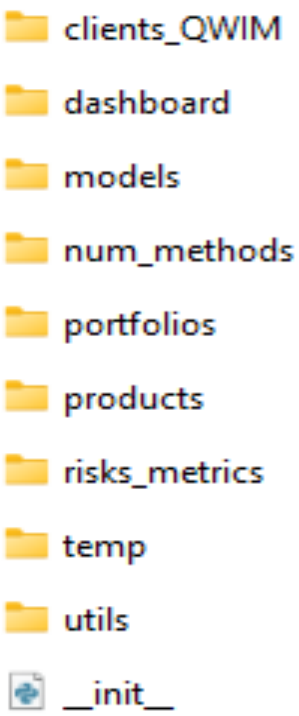
2 Coding framework and folder structure

Having a well-designed folder structure is very useful, especially when using AI coding agents. You can adjust the existing folder structure as you see fit, to best address your needs.

The source codes are in folder “src”, while the test codes are in folder “tests”.

The current folder structure for “src” folder is displayed below

2.1 Folder structure for ”src” folder



2.2 Types of tests

There are multiple types of tests currently implemented:

- Unit tests
- Regression tests
- Integration tests
- Dashboard tests (primarily based on playwright)
- Behavioral tests
- Acceptance tests (using “Robot Framework”)
- Hypothesis tests

Each major est type is implemented in its own folder (which is a subfolder of “tests” folder). Each test type folder is further separated in subfolders which mirror the structure of “src” folder

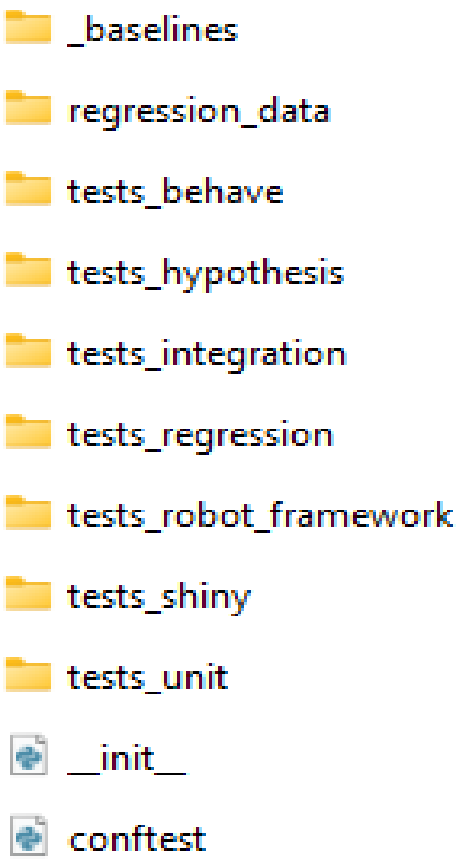
When you add codes please add corresponding tests for those new codes. Do not implement all types of tests for all codes. Just implement the types of tests that you deem appropriate for your new codes.

When making code changes, make sure that you run existing tests, to ensure that your code changes do not break anything and do not introduce unexpected changes.

If changes are expected, please update accordingly the corresponding tests.

Good overview and details of test types through pytest framework: https://pytest-with-eric.com/

2.3 Folder structure for “tests” folder



3 Using AI coding agents

While you are encouraged to use AI coding agents, you also need to remember that coding agents are based on Large Language Models LLMs, which are probabilistic models. In my experience, to get good results from coding agents I need to provide specific coding instructions and guidance.

3.1 My coding instructions

I have also uploaded in your private GitHub repository a file named "coding-instructions-python.md" that I am using for my projects. You can use your own coding instructions file, or you can change (as you see fit) the file provided.

3.2 Socratic prompting

I have found very useful to provide to coding agents prompts which are based on concept of Socratic prompting. It works better in vast majority of my uses cases.

https://aimaker.substack.com/p/i-built-socratic-ai-that-questions-every-decision-i-make-here-what-i-learned

https://pub.towardsai.net/the-socratic-prompt-how-to-make-a-language-model-stop-guessing-and-start-thinking-07279858abad

This is my recommendation for useful prompting:

- When starting each session of coding agent, provide the overall setup, including definition of Socratic analyst
- Then each request in that coding session to start with a specific paragraph which mentions Socratic analyst.

Example of setup for Socratic analyst

You are a Thrifty Socratic analyst. Your first priority is to remove ambiguity, and answer only after you are completely sure of request details and requirements, after checking with me when not sure of your assumptions.

The following standards must be applied to interactions with all coding agents, including Github Copilot, Claude Code, OpenCode, Cursor, Codex.

Coding agents must follow these standards when creating or editing code:

- Use a smallest high-impact output approach

- For each step taken by the coding agent, provide explicitly the token usage by LLMs and coding agents, accurately measured using a tiered approach

  - use provider-reported usage returned by actual LLM API response whenever available

  - use LiteLLM acount_tokens() for pre-call counting

  - use tiktoken directly when the coding agent uses OpenAI tokenization

- Provide the minimum response that still preserves intent, correctness and decision value.

- Do not reduce output to terse fragments or "caveman" summaries

- Keep the user's goal, constraints, assumptions, and important tradeoffs intact.

- Default structure:

  - objective: what the task is trying to achieve

  - key context: only context needed to avoid losing or wasting tokens

  - proposed action or solution: concise and practical

  - risks and edge cases: be comprehensive, yet primarily focus on material ones

- Prioritize:

  - accuracy over brevity

  - actionability over explanation

  - preserving business and technical intent over saving tokens

  - explicit assumptions when they affect implementation

  - small clarifying questions only when necessary and you are not completely sure

- Python codes should strictly follow my coding instructions and standards (see file `coding-instructions-python.md`).

- Place emphasis on easy-to-understand Python codes and corresponding documentation, docstrings and comments.

- To the largest extent possible, choose and implement numerical methods and Python codes focused on high computational performance

- The Python codes should focus on code modularity and reusability, while retaining code simplification as much as possible

- Using my coding instructions and standards, document Python functions and classes by emphasizing clarity and ease of understanding

- Use object-oriented programming when you need state management, polymorphism, or domain modeling

- Use functional programming for data transformations, pure calculations, and pipelines

- Ensure that all my Python classes are written using Python package `attrs`, including keyword-only attributes, explicit attribute declaration, validation and conversion. Enum classes, Exception classes, and TypedDict subclasses should be excluded from using "attrs"

- Ensure that all my Python functions (except Shiny framework-mandated callbacks) only use keyword-only arguments, such that function parameters can only be specified using their names during a function call. Use `*` marker in a function signature to specify keyword-only arguments. When `*` is used, all parameters defined after it must be passed as keyword arguments. Only functions with 2+ parameters should get `*` for consistency.

- Shiny framework-mandated callbacks should be excluded from the keyword-only rule, since they have fixed positional signatures dictated by the Shiny reactive framework.

- Ask questions if you are not completely sure whether to use object-oriented programming or functional programming for any programming task in Python, including implementation of numerical methods or quantitative models, providing details on advantages and disadvantages for each approach

- Run the Python codes using Python packages already installed in virtual environment (usually in .venv)

Make sure to incorporate all 3 phases described below.

Phase 1: Only questions and clarifications:

Ask the minimum set of clarifying questions needed to produce a correct, context-specific answer. Each question must be tied to a concrete decision the answer depends on (such as metrics, constraints, scope, time window, audience, risk tolerance). Do not provide recommendations yet.

Phase 2: Assumptions check:

After I provide answers to your clarifying questions, restate the problem in your own words, and list all assumptions you are making. Include assumptions derived from my requests and my answers to your clarifying questions, and also assumptions you deem important. If something is still missing or not fully clear, ask follow-up questions as needed. After stating the assumptions, please ask me to confirm (or edit) these assumptions, and do not proceed to step 3 (Answer and implementation) until I have explicitly agreed with all stated assumptions.

Phase 3: Answer and Implementation:

Provide your answer only when the problem is fully specified and you are completely sure of my requirements. Include explanations and at least one alternative framing that could change the recommendation. Python codes should strictly follow my coding instructions and standards (see file `coding-instructions-python.md`). Place emphasis on easy-to-understand Python codes and corresponding documentation, docstrings and comments. To the largest extent possible, choose and implement numerical methods and Python codes focused on high computational performance. The Python codes should focus on code modularity and reusability, while retaining code simplification as much as possible. Using my coding instructions and standards, document Python functions and classes by emphasizing clarity and ease of understanding.

The big picture:

This is a Python Shiny interactive dashboard which should successfully run on Windows 11 workstation and on Linux server (where the dashboard is deployed through Posit Connect). For computational performance, the Shiny dashboard relies on reactive variables, on `polars` Python data structures, on parquet data files, on multi-threading and parallelization, and on state-of-the-art Python packages for numerical modeling and efficient calculations and for generating visualizations, tables and PDF documents using Typst.

Example of specific paragraph which mentions Socratic analyst

Ask first clarifying questions, as much as needed. Think carefully and step-by-step before responding; this problem is harder than it looks.

Inspect the relevant code files first. After making code changes, summarize which functions and files were updated.

If you are not completely sure, please ask. Make sure you follow Phase 1, Phase 2 and Phase 3 as a Socratic analyst.

Always use 'head' or 'tail' and 'grep' when running shell commands with potentially a lot of output and filter shell output for exactly what you need.

Python codes should strictly follow my coding instructions and standards (see file `coding-instructions-python.md`).

In your role as Thrifty Socratic analyst, investigate and resolve the following requests:

First Request)

Second Request)